Système de Gestion de Flotte

Master 1 GIL - Université de Rouen - 2025/2026

Encadrants : Lydia & Luc

Plan de présentation

01

Microservices

Véhicules, Conducteurs, Maintenance, Localisation, Événements

02

Environnements et CI/CD

Docker, Kubernetes, Helm, GitHub Actions

03

Micro Frontends

Shell, drivers, location, maintenance, vehicles

04

Observabilité

Prometheus, Grafana, OTel, Jaeger, Loki, Promtail

05

Tests et Qualité

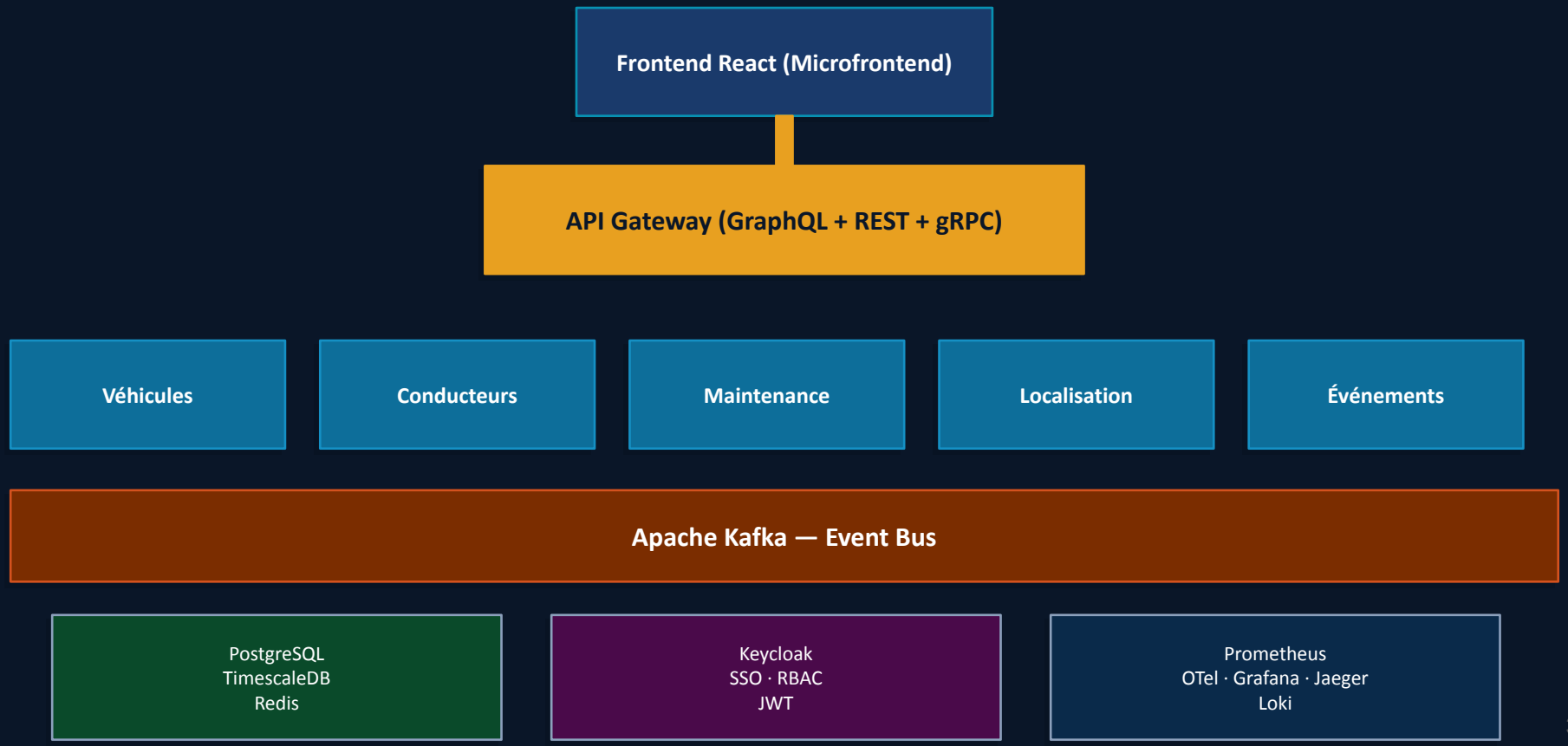
Charge, bout en bout, unitaire et intégration, linting, bruno

06

Démonstration

Scénarios live, métriques temps réel

Architecture Microservices



Services Développés

Véhicules

Spring Boot · Java 21

- Gestion de l'inventaire
- Suivi de l'état
- Assignments

Conducteurs

Spring Boot · Java 21

- Profils conducteurs
- Gestion des permis
- Disponibilité

Maintenance

Spring Boot · Java 21

- Historique d'entretien
- Planification
- Suivi de l'intervention

Événements

Spring Boot · Java 21

- Consommation Kafka
- Génération d'Alertes
- Notifications Temps Réel

Localisation

gRPC · NestJS · TimescaleDB

- Ingestion GPS en temps réel
- Streaming vers les clients
- Détection d'anomalies

Environnements

Déploiement & Orchestration

Docker

Développement Local

Docker Compose

Conteneurisation des services

Namespaces

Isolation

dev · staging · prod

Quota de ressources

Helm Charts

Packaging

Versionning

Rollback automatique

ConfigMaps & Secrets

Configuration

Externalisation

Secrets chiffrés (SOPS)

Ingress

Exposition

Routing HTTP

TLS & Load Balancing

Pipeline CI/CD — GitHub Actions

1. Tests Services Java

Spring Boot (JDK 21)

- PostgreSQL & Kafka éphémères
- Initialisation DB auto
- ./mvnw clean test

2. Tests Services Node.js

NestJS (Node 20)

- Checkout & Setup Node
- npm ci (installation propre)
- npm test (location-service)

3. Linting Helm

Infrastructure as Code

- Validation fleet-infra/app/obs
- helm lint (fichiers K8s)

4. Build & Push — ghcr.io

Condition : Main/Staging + Succès jobs 1, 2, 3

- **Multi-plateforme :** Setup QEMU & Buildx
- **Authentification :** GitHub Container Registry (ghcr.io)
- **Itération multi-services :** drivers, maintenance, vehicles, events, location, gateway, frontend
- **Versionnement :** Tags :latest et :${{ github.sha }}
- **Optimisation :** Utilisation du cache Actions pour des builds rapides

✅ *Pipeline robuste : Zéro image si tests ou infra invalides.*

Architecture Micro Frontends

Composition & Orchestration (Nx Monorepo)

Shell (Host)

Application Socle

- Point d'entrée unique
- Navigation principale
- Sécurité (Keycloak)
- Module Federation

Remote Apps (Micro-frontends)

drivers | location | maintenance | vehicles

Technologie : Webpack/Vite Module Federation

Observabilité

📊 Observabilité — 3 Piliers

Ingestion et Collecte

- OpenTelemetry Collector : centraliser les données et redistribuer
- Promtail : scrapper les logs

Logs

- Loki : stocker les événements (logs)

Tracing

- Jaeger : suivre les requêtes utilisateur

Visualisation

- Grafana : Interface web

Métrique

- Prometheus : stocker les métriques de performances

Tests et Qualité

✅ Tests Unitaires & Intégration

JUnit 5 + Mockito (Spring Boot)

80% couverture (JaCoCo)

Jest + Supertest (NestJS)

80% couverture (Istanbul)

Tests d'Intégration

- Testcontainers (BDD + Kafka réels)
- MockMvc et Supertest
- API et contrats inter-services vérifiés

Couverture globale

80% mesurée

🌐 E2E, Charge & Qualité CI

End-to-End — Playwright

- Parcours complets : Auth → CRUD
- Bloquant avant déploiement

Tests de Charge — k6

- 500 véhicules simultanés (P95 < 200ms)
- Analyse latence par endpoint

Résultats & Qualité CI

- Badge de couverture dans chaque PR
- Cycle : lint → test → build → deploy

Merci de votre attention